Korambayil Ahamed Haji Memorial

UNITY WOMEN'S COLLEGE

Govt. Aided & Affiliated to University of Calicut

Accredited by NAAC with CGPA 3.20 & A Grade

BudgetBuddy

A PROJECT REPORT

Submitted by

MANEESHA K(UTAXSCS017)
FATHIMA SHABEEBA (UTAXSCS011)
RANA SHIRIN VM (UTAXSCS042)
SHAMNA T (UTAXSCS029)

Under the Guidance Of

Dr. Haris U
Assistant Professor & Head
PG Department Of Computer Science

IN PARTIAL FULFILMENT OF THE REQUIRMENTS FOR THE AWARD OF

BACHELOR OF COMPUTER SCIENCE

University of Calicut

March 2026

PG DEPARTMENT OF COMPUTER SCIENCE

KAHM UNITY WOMEN'S COLLEGE

P.O. NARUKARA, MANJERI 676122

Korambayil Ahamed Haji Memorial

UNITY WOMEN'S COLLEGE

Govt. Aided & Affiliated to University of Calicut

Accredited by NAAC with CGPA 3.20 & A Grade

CERTIFICATE

Project Certificate for B.Sc. Computer Science Students

Certified that this project titled **BudgetBuddy** is the bonafide work of Ms. Maneesha K (Register No:UTAXSCS017), Ms. Fathima Shabeeba (Register No:UTAXSCS011), Ms. Shamna T (Register No:UTAXSCS029), Ms. Rana Shirin VM (Register No:UTAXSCS042) of the **Sixth Semester B.Sc. Computer Science** who carried out the work under my supervision in partial fulfillment for the **Degree of Bachelor of Science** during the academic year **2023 - 2026**. Certified further that to the best of my knowledge this report had not been submitted for any other examination and does not form any other course undergone by the candidate.

Dr. Haris U
Assistant Professor & Head
PG Department of Computer Science

Date:

Place: Manjeri

Internal Examiner External Examiner

"ENTRY TO THE NEXT PHASE"

REGIONAL TECHNOLOGIES

regionaliit@gmail.com

Ref:

Date: 17/02/2026

CERTIFICATE OF COMPLETION

This is to certify that **Fathima Shabeeba** (Reg. No: UTAXSCS011), **Maneesha K** (Reg. No: UTAXSCS017), **Rana Shirin VM** (Reg. No: UTAXSCS042), **Shamna T** (Reg. No: UTAXSCS029) have successfully completed their project entitled "**BudgetBuddy**" using Python (Django) & Flutter under the guidance of our senior developers during the period from August 2025 to December 2025.

During this period they were found to be hardworking, punctual & efficient. We wish them success in their future endeavors.

For REGIONAL TECHNOLOGIES

CEO

REGIONAL TECHNOLOGIES CALICUT OPP. NEW BUS STAND

Abishek Corner, Puthiyambalam Road, Near Indoor Stadium, Opposite New Bus stand, Calicut-4

Ph: 0495 6999778, 9526848922

DECLARATION

We wish to state that the work embodied in this project titled **BudgetBuddy** forms our own contribution to the project work carried out under the guidance of Dr. Haris U, Assistant Professor & Head, Department of Computer Science. Also we here by declare that this work is submitted to University of Calicut in fulfillment of requirement for the award of the degree of Bachelor of Computer Science. And this work has not been submitted for any other degree of this or any other University. Wherever references have been made to previous work of others, it has been clearly indicated as such and included in the Bibliography.

Maneesha K
Reg. No: UTAXSCS017

Fathima Shabeeba
Reg. No: UTAXSCS011

Rana Shirin VM
Reg. No: UTAXSCS042

Shamna T
Reg. No: UTAXSCS029

Place: Manjeri

Date:

ACKNOWLEDGEMENT

We would first like to thank the Almighty Allah who created us, taught us by pen and has been teaching us everything that we do not know, and in turn for giving us the courage and the perseverance in completing this project work. This project itself is an acknowledgement for all those who have given us their heart- felt-cooperation in making it a grand success.

We are thankful to our principal, Prof. (Dr.) Muhammed Basheer Ummathur for providing the necessary infrastructure and labs. We are greatly indebted to our supervisor, Dr. Haris U, for valuable guidance at every stage of this project work.

We are also thankful to project coordinator Dr. Haris Ummath for extending his sincere guidance throughout this work, without his supervision and many hours of devoted guidance, simulating and constructive criticism, this would never come out in this form.

We extend our gratitude to our team members, Maneesha K, Fathima Shabeeba, Shamna T, Rana Shirin VM whose dedication, teamwork, and hard work made this project a success. The collaboration and collective efforts played a crucial role in achieving our objectives.

The last but not the least, we would like to express my deep and earnest thanks giving to our parents for their moral support and heartfelt cooperation in doing the project.

ABSTRACT

BudgetBuddy is an intelligent expense management and budgeting application designed to help individuals efficiently track their income and daily expenses. The system provides a smart platform to monitor spending patterns, manage financial goals, and improve budgeting discipline through predictive forecasting and personalized suggestions.

With distinct modules such as Admin, Expert, and User, BudgetBuddy enables seamless interaction and financial guidance. Users can easily add and categorize their transactions, set budget limits, receive alerts when expenses exceed predefined thresholds, and generate detailed expense reports. The system also supports goal setting and reminders for better financial planning.

Additionally, BudgetBuddy integrates forecasting techniques using historical transaction data to predict future expenses, offering proactive insights and expert recommendations. Experts can provide useful tips, suggestions, and communicate with users through chat support, while the admin manages accounts, complaints, feedback, and overall system monitoring.

BudgetBuddy acts as a complete digital companion for expense tracking, forecasting, and smart financial decision-making, ensuring sustainable budgeting and improved financial awareness among users.

CONTENTS

CHAPTERS

1. INTRODUCTION 9

1.1 OVERVIEW OF THE PROJECT 9
1.2 PROBLEM STATEMENT 9
1.3 OBJECTIVES OF THE PROJECT 10
1.4 SCOPE OF THE PROJECT 10
1.5 EXPECTED OUTCOMES 11

2. SYSTEM ANALYSIS 13

2.1 EXISTING SYSTEM 13
2.2 PROPOSED SYSTEM 14

3. FEASIBILITY STUDY 15

3.1 TECHNICAL FEASIBILITY 15
3.2 ECONOMICAL FEASIBILITY 15
3.3 OPERATIONAL FEASIBILITY 16
3.4 BEHAVIORAL FEASIBILTY 16

4. METHODOLOGY 17

4.1 SELECTION OF THE AGILE MODEL 17
4.2 APPLICATION OF AGILE IN BUDGETBUDDY 18
4.3 SCRUM FRAMEWORK 18

5. SYSTEM REQUIREMENTS 19

5.1 HARDWARE REQUIREMENTS 19
5.2 SOFTWARE REQUIREMENTS 19
5.3 FRONTEND 20
5.4 BACKEND 21
5.5 FLUTTER (MOBILE APPLICATION) 22

6. SYSTEM DESIGN 25

6.1 INPUT DESIGN 25
6.2 OUTPUT DESIGN 27
6.3 NORMALIZATION 28
6.4 MODULES DESCRIPTION 29
6.5 ENTITY RELATIONSHIP (ER) DIAGRAM 30
6.6 DATA FLOW DIAGRAM 30
6.7 DATABASE DESIGN 34
6.8 DATABASE TABLES 34

7. SYSTEM TESTING & IMPLEMENTATION 39

7.1 SYSTEM TESTING 39
7.2 SYSTEM IMPLEMENTATION 40

8. SYSTEM MAINTENANCE 43

8.1 CORRECTIVE MAINTENANCE 43

9. FUTURE ENHANCEMENT 45

10. CONCLUSION 47

11. BIBLIOGRAPHY **49**

12. APPENDIX **51**

12.1 SAMPLE SCREENSHOTS 51
12.2 SAMPLE CODES 54

1. INTRODUCTION

This study develops a predictive model to forecast people expenses using historical data and advanced forecasting techniques. The model integrates several functions to track and manage expenses effectively. These functions include adding daily expenses, which accumulate to provide a comprehensive total, and calculating total expenses over specified periods. The model also predicts future expenses based on historical data, generating detailed reports to facilitate financial analysis and decision-making. Additionally, the model allows for the categorization of expenses and provides financial advice. Users can set budget alerts to receive notifications when expenses approach or exceed predefined limits. By implementing these tools, the model aims to enhance financial planning and budgeting for students and administrators, fostering better financial management and sustainability.

1.1 Overview of the Project

BudgetBuddy is a smart financial management application designed to assist users in tracking their income, expenses, and budgeting activities digitally. The system allows users to record transactions, categorize spending, and generate detailed reports for better financial awareness.

The platform also includes key features such as budget limit alerts, expense forecasting, goal setting, and reminders. Additionally, BudgetBuddy provides expert support through suggestions and chat interaction, making it a complete personal finance companion.

The system consists of three major roles:

- **Admin:** Manages accounts, reports, and user feedback
- **Expert:** Provides financial tips and recommendations
- **User:** Tracks expenses, sets goals, and receives alerts

1.2 Problem Statement

Many individuals face difficulties in managing personal finances due to the lack of an organized expense tracking system. Manual methods of recording expenses are time-consuming and often inaccurate, resulting in poor budgeting and financial stress.

Users also lack automated alerts, reminders, and forecasting features that could help them control overspending and plan future expenses.

Therefore, there is a need for a smart and user-friendly system like BudgetBuddy, which automates expense tracking, provides insights, and supports better financial planning.

1.3 Objectives of the Project

The main objectives of BudgetBuddy are:

1. To automate income and expense tracking digitally
2. To categorize transactions for better financial analysis
3. To provide budget monitoring with alerts for overspending
4. To support savings goal setting and progress tracking
5. To generate expense reports and future spending forecasts
6. To provide reminders for payments and planned expenses
7. To offer expert suggestions and financial guidance
8. To ensure secure storage of user and transaction data
9. To develop a simple and user-friendly interface
10. To support scalability for future enhancements

1.4 Scope of the Project

The scope of BudgetBuddy includes:

- Digital management of income and expenses
- Budget limit setting and alert notifications
- Financial goal planning and reminders
- Report generation and expense forecasting
- Expert interaction through tips and chat support
- Admin control for managing experts, view feedback, and complaints

The system is designed for individuals and students and can be enhanced further with AI-based recommendations and banking integrations.

1.5 Expected Outcomes

The expected outcomes of BudgetBuddy are:

- Improved financial awareness and budgeting discipline
- Reduction in manual effort through automated expense tracking
- Better control over spending with alerts and reminders
- Effective savings goal achievement support
- Forecast-based planning for future expenses
- Secure storage and organized financial reporting
- Enhanced user convenience with expert financial guidance
- Scalability for future advanced financial features

2. SYSTEM ANALYSIS

2.1 Existing System

Managing personal expenses is commonly carried out using manual methods such as notebooks, spreadsheets, or simple budgeting applications. Individuals typically record their daily expenditures, categorize them into different spending groups, and compare their expenses against predefined budgets. Several open-source expense management applications such as Wallet Expense Tracker, MMAS Money Tracker, and Finance App Flutter available on GitHub provide basic features for recording and monitoring financial transactions (Dreamcatcher45, 2021); (Nguyen, 2022). These systems allow users to add income and expenses, categorize transactions, and generate simple financial summaries to help track spending activities.

However, most existing systems rely heavily on manual data entry, which can be time-consuming and may lead to errors or incomplete records. Additionally, many basic budgeting tools focus primarily on recording past transactions and providing simple summaries, without offering deeper insights into spending behaviour (Santos, 2021). They often lack advanced functionalities such as predictive analysis, automated financial recommendations, goal-based financial planning, and intelligent alerts based on spending patterns. As a result, individuals may find it difficult to clearly understand their financial habits or make informed financial decisions.

Furthermore, traditional systems do not always integrate multiple financial aspects within a single platform, which limits their ability to provide a comprehensive view of personal finances. Because of these limitations, users often require a more intelligent and user-friendly solution that can support better financial planning and expense management.

2.2 Proposed System

The proposed system aims to enhance expense management by incorporating advanced predictive analytics and comprehensive financial tracking. It features automated daily expense tracking, where students can easily log their expenditures, which are then categorized and totaled. The system uses historical data to predict future expenses, offering personalized financial advice and generating detailed visual reports of spending trends. It includes budget setting and alert functions to notify users when they approach or exceed their set limits. Additionally, the system integrates various financial data sources, providing a complete and accurate view of an individual's financial situation. This comprehensive approach enables better financial planning, proactive management, and sustainable budgeting for individuals and organizations.

3. FEASIBILITY STUDY

Prior to the commencement of development, the BudgetBuddy project team conducted a structured feasibility study to determine whether the proposed intelligent expense management system could be successfully designed, built, and deployed within the constraints of the academic project timeline, available resources, and target user environment. The study was carried out during the project initiation phase and involved direct assessment activities including team capability review, technology stack evaluation, prototype testing, informal user interviews with fellow students, and cost estimation exercises. The findings of the study across four feasibility dimensions are documented in this chapter.

3.1 Technical Feasibility

The development team conducted a technical feasibility assessment to evaluate whether the available hardware, technology stack, and team expertise were sufficient to develop and deploy BudgetBuddy. The assessment included a review of the selected technologies, a small-scale prototype implementation, and a hardware audit.

The review confirmed that Flutter, along with a Python Django backend and MySQL database, could support all planned features such as budget alerts, expense forecasting, multi-role authentication, and chat functionality. The prototype successfully demonstrated data transfer between the mobile app and backend system. The hardware audit verified that development machines and Android testing devices met the required specifications. No technical limitations were identified, and the project was deemed technically feasible.

3.2 Economical Feasibility

The team conducted a cost-benefit analysis to evaluate whether BudgetBuddy could be developed within academic financial constraints and whether its benefits justified the resources invested. Development costs were minimal, as the entire software stack—Flutter, Python, Django, MySQL, Android Studio, PyCharm, and VS Code—was available under free or open-source licenses. No additional hardware was required. Estimated backend hosting costs for small-scale deployment ranged from ₹300–₹500 per month, which was considered affordable. No expenses were incurred during the academic development phase.

On the benefits side, the system provides direct advantages such as automated expense tracking, budget monitoring, and time savings in financial reporting. Indirect benefits include improved financial awareness and responsible spending habits among students. Since development costs were negligible and the potential user benefits were significant, the team concluded that BudgetBuddy is economically feasible and cost-effective.

3.3 Operational Feasibility

To evaluate operational feasibility, the team conducted structured interviews with twelve students from the target user group. Most participants either tracked expenses manually or not at all due to inconvenience. All expressed willingness to use a simple and user-friendly mobile application. Key requested features included automated budget alerts and report generation, which are core components of BudgetBuddy.

A paper-based walkthrough of the proposed interface was also conducted with selected participants, who navigated the system flow without difficulty and provided positive feedback. The administrative workflow was reviewed with a faculty supervisor and found to be appropriate and manageable. As no major operational issues were identified, the team concluded that BudgetBuddy is operationally feasible and well-aligned with user needs.

3.4 Behavioral Feasibilty

To assess behavioural feasibility, the team conducted an attitudinal survey among twenty students. Most respondents were comfortable using mobile applications for personal management, and fifteen indicated willingness to adopt BudgetBuddy if it was reliable and easy to use. Minor concerns about data privacy were addressed through secure login, password protection, and OTP verification.

The majority of participants found expert financial tips and personalised suggestions useful, supporting the inclusion of the Expert module. A simulated onboarding test showed that new users could navigate key features without difficulty. Based on these findings, the team concluded that BudgetBuddy is behaviourally feasible and likely to achieve strong user acceptance.

4. METHODOLOGY

Every software project requires a structured Software Development Life Cycle (SDLC) model to guide planning, development, and delivery. For the BudgetBuddy project, the team evaluated the Waterfall, Spiral, and Agile models before selecting Agile with the Scrum framework. Agile was chosen due to its flexibility, iterative development process, and suitability for small, cross-functional teams working under fixed academic deadlines.

4.1 Selection of the Agile Model

Agile was selected because BudgetBuddy's requirements evolved during development. While core features such as expense tracking and budget alerts were predefined, modules like forecasting and expert chat required refinement based on user feedback. Unlike Waterfall, Agile allowed continuous requirement updates without disrupting progress.

Additionally, Agile ensured that a working version of the system was delivered at the end of each sprint, enabling regular supervisor reviews and early issue detection. The Spiral model was considered but rejected due to its higher process complexity, which was unnecessary for a four-member academic team.

Graphical illustration of Agile Model:

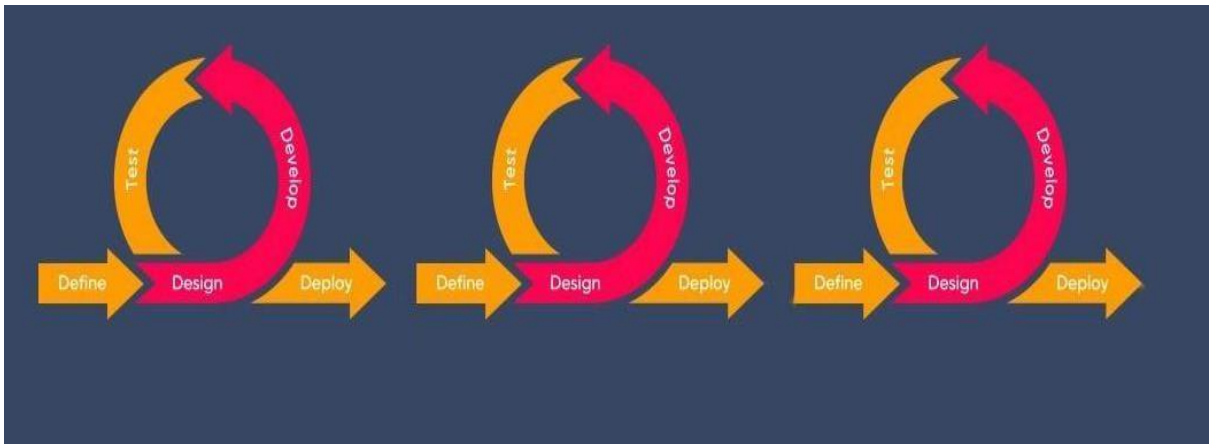

Figure 4.1: Agile Development Cycle as Applied in BudgetBuddy

4.2 Application of Agile in BudgetBuddy

The development followed time-boxed sprints. A Product Backlog was created listing all system features in priority order. Before each sprint, selected items were moved into a Sprint Backlog during Sprint Planning. At the end of each sprint, a working increment of BudgetBuddy was demonstrated to the supervisor for feedback.

Daily stand-up meetings were conducted to monitor progress and resolve blockers. Sprint Reviews and Retrospectives ensured continuous improvement and smooth integration of new features.

Sprint Summary

- Sprint 1: Project setup, backend configuration, authentication module.
- Sprint 2: Admin module development.
- Sprint 3: Expert module implementation.
- Sprint 4: User mobile application (income/expense, goals, reminders).
- Sprint 5: Forecasting, reports, and chat integration.
- Sprint 6: Testing, bug fixing, UI refinement, and final deployment.

Each sprint delivered a tested and functional system increment.

4.3 Scrum Framework

Within Agile, the Scrum framework was adopted to structure development activities. Scrum provided clear roles, defined ceremonies, and regular delivery cycles, preventing scope creep and ensuring accountability.

Scrum Roles

Product Owner: Project supervisor (requirement validation and prioritisation).

Scrum Master: Rotating team member facilitating Scrum practices.

Development Team: Four members responsible for design, coding, testing, and integration.

Scrum Artefacts

- **Product Backlog:** Complete list of features and requirements.
- **Sprint Backlog:** Selected tasks for each sprint.
- **Increment:** Working version of BudgetBuddy delivered at the end of each sprint.

5. SYSTEM REQUIREMENTS

5.1 Hardware Requirements

The selection of hardware is very important in the existence and proper working of any of the software. When selecting hardware, the size and capacity requirements are also important. The hardware must suit all application developments.

Processor	: i3 or above
System Bus	: 32 Bit or 64 Bit
RAM	: 4GB or Above
HDD	: 500 GB or Above
Monitor	: 14” LCD or Above
Keyboard	: 108 Keys
Mouse	: Any Type of mouse
Mobile	: Android supported mobile phone

5.2 Software Requirements

One of the most difficult tasks is selecting software, once the system requirement is find out then we have to determine whether a particular software package fits for those system requirements. This section summarizes the application requirement.

Operating System	: Windows 7or above
Technologies used	: Android
Front end	: Html, CSS, Bootstrap, JS
Back End	: Python-Django or Flutter/Java
Database	: Mysql
IDE	: Android studio, PyCharm professional, VS code
Web browser:	: Chrome, Explorer, Edge, etc…

5.3 Frontend

The frontend of the BudgetBuddy Expense Management System is the part of the application responsible for the user interface (UI) and user experience (UX). It plays a crucial role in enabling users to easily interact with the platform and access features such as tracking income and expenses, viewing reports, setting financial goals, receiving alerts, and communicating with experts.

UI/UX Design

A clean, intuitive, and user-friendly interface was designed to ensure smooth interaction with the BudgetBuddy system. The User module is implemented as a mobile application, allowing individuals to easily track expenses, set goals, and receive alerts on the go.

Meanwhile, the Admin and Expert modules are developed as a web-based interface, enabling efficient monitoring, report management, and expert guidance through a larger desktop-friendly environment.

Technologies Used

- **HTML:** Used for structuring the web pages and forms.
- **CSS:** Applied for styling and layout customization.
- **Bootstrap:** Used to design responsive and visually appealing interfaces with pre-built UI components such as buttons, forms, grids, and alerts.
- **JavaScript:** Enables interactive features and supports dynamic behaviour through Bootstrap components.

Features

- **Transaction Entry Interface:** Users can easily add income and expense records through a simple form with category selection and date input.
- **Expense Dashboard:** Provides a clear summary of spending patterns with charts and categorized breakdowns.
- **Reports Display:** Users can view detailed expense reports, monthly summaries, and forecasting results in an organized format.
- **Goal & Reminder Interaction:** The interface allows users to set savings goals, schedule reminders, and receive alerts when nearing budget limits.

5.4 Backend

The backend of the BudgetBuddy System is responsible for handling server-side operations such as processing user requests, managing authentication, storing transaction data, generating forecasts, and ensuring secure communication between modules.

Server-Side Logic

The backend processes essential operations such as:

- Recording income and expense transactions
- Retrieving financial reports and summaries
- Sending budget alerts and reminders
- Handling expert-user communication
- Managing feedback and complaint submissions

It also ensures system security through authentication, session handling, and data validation.

Technologies Used

- **Python:** The primary backend programming language due to its flexibility and strong support for data analysis and forecasting.
- **Django:** Python framework used for developing the backend. Suitable for scalable applications with built-in modules such as authentication, ORM, and admin panel support.

Database Management

The backend communicates with a database that stores:

- User profiles and login credentials
- Transaction history (income and expenses)
- Budget goals and reminders
- Expert tips and suggestions
- Reports, feedback, and complaints

A database system such as **MySQL** is used for reliable and structured storage of financial data.

Forecasting and Analytics Integration

BudgetBuddy integrates forecasting techniques to predict future expenses based on historical transaction data. The backend interacts with analytical models that generate:

- Future expense trends
- Spending pattern insights
- Personalized budgeting suggestions

These predictions help users plan their finances more effectively and avoid overspending.

5.5 Flutter (Mobile Application)

To ensure users can manage their expenses anytime and anywhere, a mobile application for BudgetBuddy was developed using **Flutter**. Flutter provides a cross-platform solution, allowing the application to run smoothly on both Android and iOS devices, increasing accessibility and convenience.

Why Flutter?

Flutter was chosen due to its ability to provide native-like performance while maintaining a single codebase for multiple platforms. This reduces development time and ensures uniform design across devices. Flutter also offers a rich collection of widgets and strong community support.

Technologies Used

- **Dart:** The programming language used to develop Flutter applications.
- **Flutter SDK:** The framework for building cross-platform mobile apps with responsive UI components.

Mobile Application Features

- **Expense Entry:** Users can quickly add daily expenses and income through the mobile interface.
- **Budget Alerts:** The app sends notifications when spending approaches or exceeds the set budget.

- **Goal Tracking:** Users can monitor progress toward savings goals and view financial achievements.
- **Reports & Forecasting:** Monthly expense reports and predicted future expenses are displayed in an easy-to-understand format.
- **User Profile & History:** Users can access their transaction history, reminders, and expert suggestions anytime.

Cross-Platform Benefits

By using Flutter, BudgetBuddy works seamlessly on both Android and iOS devices. This eliminates the need for separate native applications, making updates and enhancements easier while ensuring consistent performance across platforms.

6. SYSTEM DESIGN

The system design phase of BudgetBuddy translated the requirements established during analysis into a concrete blueprint for construction. The design effort addressed four interconnected concerns:

- How users would interact with the system through well-structured input screens (Input Design).
- What information the system would produce and in what form (Output Design).
- How the underlying data would be organized and stored without redundancy (Normalization).
- How the system's functionality would be divided across distinct, manageable components (Modules Description).

Each of these design decisions was made with the project's three user roles — Admin, Expert, and User — as the primary frame of reference, ensuring that every design choice traced directly back to a real user need identified during the requirements phase.

6.1 Input Design

Input design for BudgetBuddy focused on defining the precise set of data entry points through which each user role would supply information to the system, and on establishing the validation rules that would govern those inputs. The design principle adopted was minimal friction: every input screen was designed to capture only the data fields strictly necessary for the operation being performed, with no optional clutter. All input forms across both the mobile (Flutter) and web (Django) interfaces follow a consistent field-label-above-input layout, with Bootstrap's validation states providing immediate inline feedback on invalid entries without requiring a page reload.

User Module — Input Screens

The primary data entry screens in the User module are the Transaction Entry form and the Goal and Reminder setup forms. The Transaction Entry form captures five fields: Transaction Type (Credit/Debit, implemented as a dropdown to prevent free-text errors), Mode of Payment (Online/Offline, dropdown), Date (read-only field populated via the Flutter

DatePicker widget to enforce valid date formats), Expense/Income Type (free text for category description), and Amount (numeric keyboard enforced at the widget level to prevent non-numeric entry). All five fields are mandatory; the Save button remains inactive until all fields carry valid values, preventing incomplete transaction records from reaching the database.

The Goal Setup form captures the goal Amount (numeric) and a Details field (free text description), with the date auto-populated to the current date on submission. The Reminder form captures Reminder Type (monthly, date-based, or yearly — dropdown), Amount, Details, a user-selected reminder date via DatePicker, and Status, which defaults to 'Active' on creation. Registration captures Name, Date of Birth, Gender, Photo (file upload), Place, Postal code, PIN, Phone number, and Email, each validated for format and uniqueness against the existing database before the record is committed.

Expert Module — Input Screens

The Expert module's primary input screens are the Add Tip form and the Add Suggestion form, both implemented as web forms within the Django admin interface. The Add Tip form takes a Tip title (short text), a Details field (longer free text), and auto-assigns the date and the logged-in expert's ID as the Expert_id foreign key on submission. The Add Suggestion form targets a specific user (selected from a dropdown populated with the expert's assigned users), and captures the suggestion body as free text. The Chat interface accepts a plain-text Message field; the From_id and To_id fields are resolved automatically from the active session, and the timestamp is captured server-side to prevent client-side manipulation.

Admin Module — Input Screens

The Admin module's input activity is primarily limited to the Complaint Reply form, which presents the original complaint text as read-only context and provides a Reply text area in which the Admin types the response. On submission, the Reply field is written back to the corresponding record in the Complaint table and the status is updated. Expert creation is handled through the standard Django admin panel, where the Admin enters all Expert profile fields (mirroring the User table structure) and assigns login credentials. All Admin inputs are protected behind session-authenticated views; unauthenticated requests are automatically redirected to the login page.

Validation Rules Applied Across All Input Screens

Table 6.1: Validation Rules Applied Across All Input Screens

Field	Input Type	Validation Rule
Transaction Type	Dropdown	Must select one of: Credit, Debit
Amount	Numeric keyboard	Must be a positive integer; zero and negative values rejected
Date	DatePicker (Flutter)	Cannot be a future date for expense entry; enforced by picker range
Email	Text (email keyboard)	Regex validation for valid email format; uniqueness check against Auth_user
Phone number	Numeric	Must be exactly 10 digits; leading zero accepted
OTP (password reset)	Numeric	Must match the server-generated OTP within the Created_at expiry window
Reminder Type	Dropdown	Must select one of: Monthly, Date-based, Yearly
Photo (profile)	File upload	Accepts JPEG and PNG only; file size capped at 2 MB server-side

6.2 Output Design

The output design of BudgetBuddy defines how information is presented to users through multiple channels, including the Flutter mobile application, Django web interface, push notifications, and expense reports. Outputs are role-based, ensuring that Users access only their personal financial data, Experts view information related to their assigned users, and the Admin sees system-wide aggregated data. In the mobile application, the primary output screen is the Expense Dashboard, which displays monthly income and expense summaries along with a categorized transaction list, color-coded (green for credit and red for debit) for quick visual distinction. The Report screen provides monthly totals and net balance, while the Forecast screen predicts upcoming expenses based on historical data. The Goal screen shows progress percentage and remaining amount, and the Reminder screen lists active and overdue reminders. The system also generates high-priority push notifications for budget limit alerts and scheduled reminders. In the web interface, the Admin dashboard presents user and expert details and manages complaints, while the Expert dashboard displays assigned user profiles and transaction summaries to support financial guidance. Overall, the output design ensures clarity, accessibility, and role-specific information delivery within the BudgetBuddy system.

6.3 Normalization

The BudgetBuddy database was designed from first principles with normalization applied across all tables to eliminate redundancy, prevent update anomalies, and ensure data integrity. The normalization process was carried out in three stages corresponding to the First, Second, and Third Normal Forms. What follows is a demonstration of how normalization was applied to the core tables of the system, using the Transaction and User entities as primary examples.

First Normal Form (1NF)

Before normalization in BudgetBuddy, multiple transactions were stored in a single User field as comma-separated values, which violated 1NF because the data was not atomic and contained repeating groups. To resolve this, transactions were moved to a separate Transaction table, with one row per transaction linked to the User via a foreign key. This ensured that all fields store single, indivisible values. Similar restructuring was applied to Tips, Suggestions, Reminders, Chat, and Goals, eliminating redundant and concatenated data across the database.

Second Normal Form (2NF)

After achieving 1NF, BudgetBuddy ensured compliance with Second Normal Form (2NF) by confirming that all non-key attributes fully depend on their respective primary keys. Since each table uses a single-column primary key (id), partial dependency was avoided by design. Additionally, user authentication details (username and password) were separated into the Auth_user table, while profile information (Name, DOB, Gender, Photo, Place, Phone number) was stored in the User table with a LOGIN_id foreign key. This separation ensures full functional dependency and prevents unnecessary modification of profile data when login credentials are updated.

Third Normal Form (3NF)

In the final stage of normalization, BudgetBuddy eliminated transitive dependencies to satisfy Third Normal Form (3NF). For example, the Feedback table initially stored the Expert’s Name along with Expert_id, creating a transitive dependency. This was corrected by removing the Expert’s Name from the Feedback table and retrieving it through a JOIN operation using

the Expert_id foreign key. The same approach was applied across the database, ensuring that no table stores derived or indirectly dependent attributes. All descriptive information is fetched through proper foreign key relationships, preventing data duplication and ensuring consistent updates throughout the system.

6.4 Modules Description

1. ADMIN

The Admin module in BudgetBuddy is responsible for overall system management and supervision. The Admin logs into the system through secure authentication and manages Expert accounts by creating, updating, or monitoring their profiles. The Admin can view registered user details, monitor feedback submitted by users, and handle complaints by reviewing them and sending appropriate replies. Additionally, the Admin can access system-generated reports to oversee platform activity and ensure smooth operation of the application.

2. EXPERT

The Expert module enables financial experts to interact with users and provide guidance. After logging in securely, Experts can add and manage financial tips to help users improve their money management skills. They can create and manage personalized suggestions for assigned users based on their spending patterns. The module also includes a chat feature that allows direct communication between the Expert and users. Furthermore, Experts can view user details and review feedback to provide better financial advice.

3. USER

The User module is the core component of BudgetBuddy, designed to help individuals manage their personal finances effectively. Users begin by registering and logging into the system. They can view expert tips, manage income and expense transactions, set financial goals, and receive alerts based on spending patterns. The system allows users to set reminders (monthly, date-based, or yearly), access forecasting features for predicting future expenses, and generate financial reports. Users can also communicate with Experts through chat, submit feedback about the system, and send complaints while tracking replies from the Admin.

6.5 Entity Relationship (ER) Diagram

An Entity Relationship (ER) Diagram is a graphical representation of the database structure, describing the entities (tables) and the relationships between them. In the BudgetBuddy system, the ER diagram represents major entities such as User, Transaction, Goal, Reminder, Feedback, and Expert.

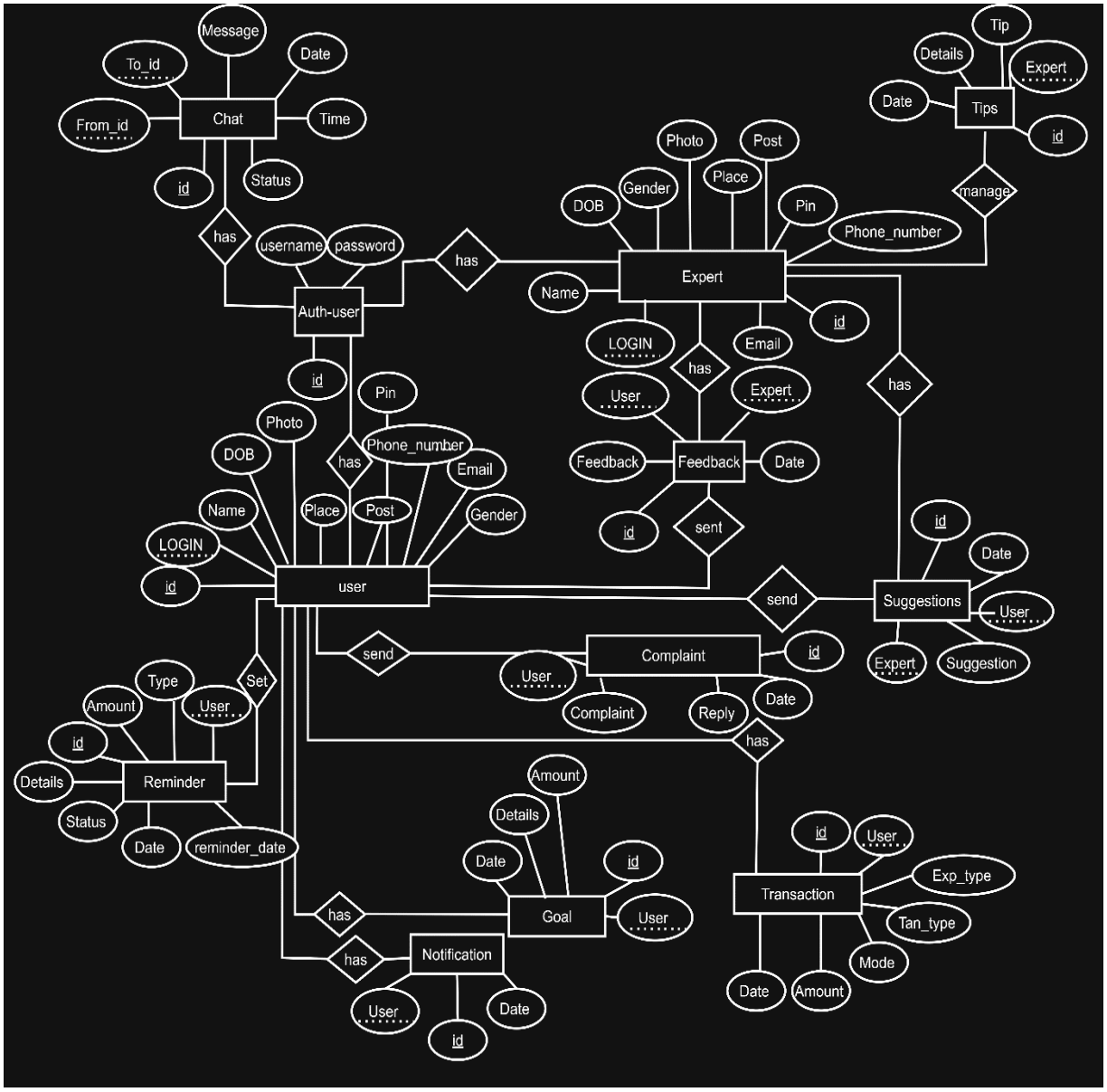

Figure 6.1: Entity Relationship (ER) Diagram of BudgetBuddy Database

6.6 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. It consists of processes (circles), data stores (parallel lines), data flows (arrows), and external entities (rectangles).

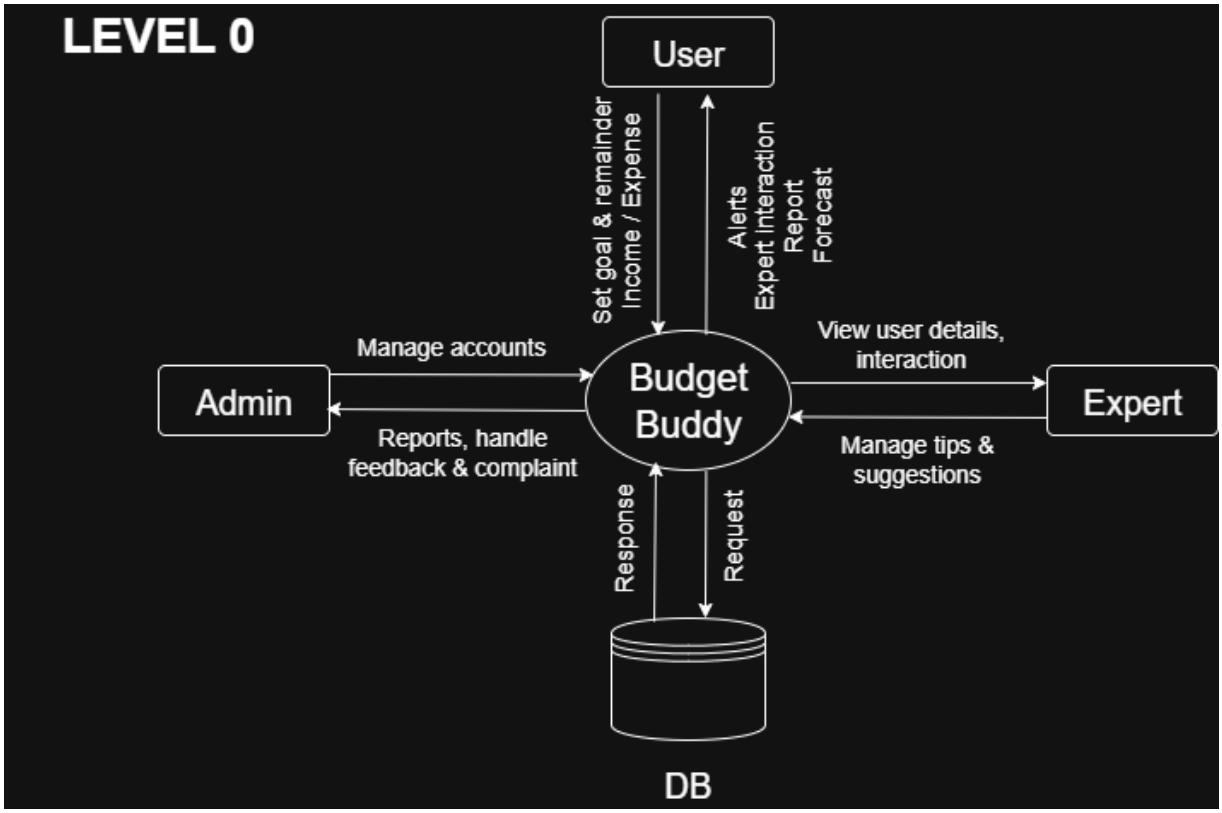

Figure 6.2: Level 0 Data Flow Diagram (Context Diagram) of BudgetBuddy

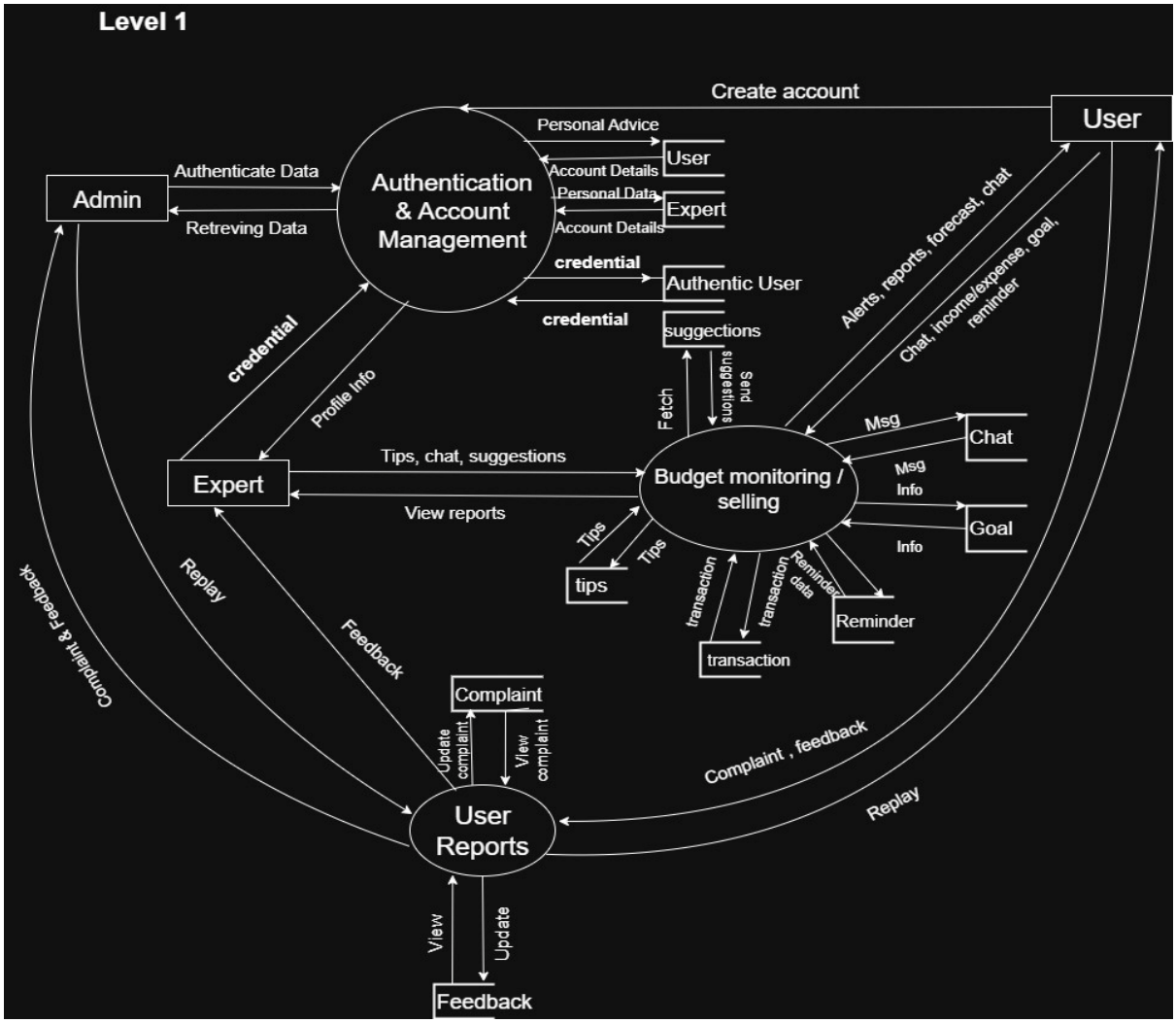

Figure 6.3: Level 1 Data Flow Diagram of BudgetBuddy System

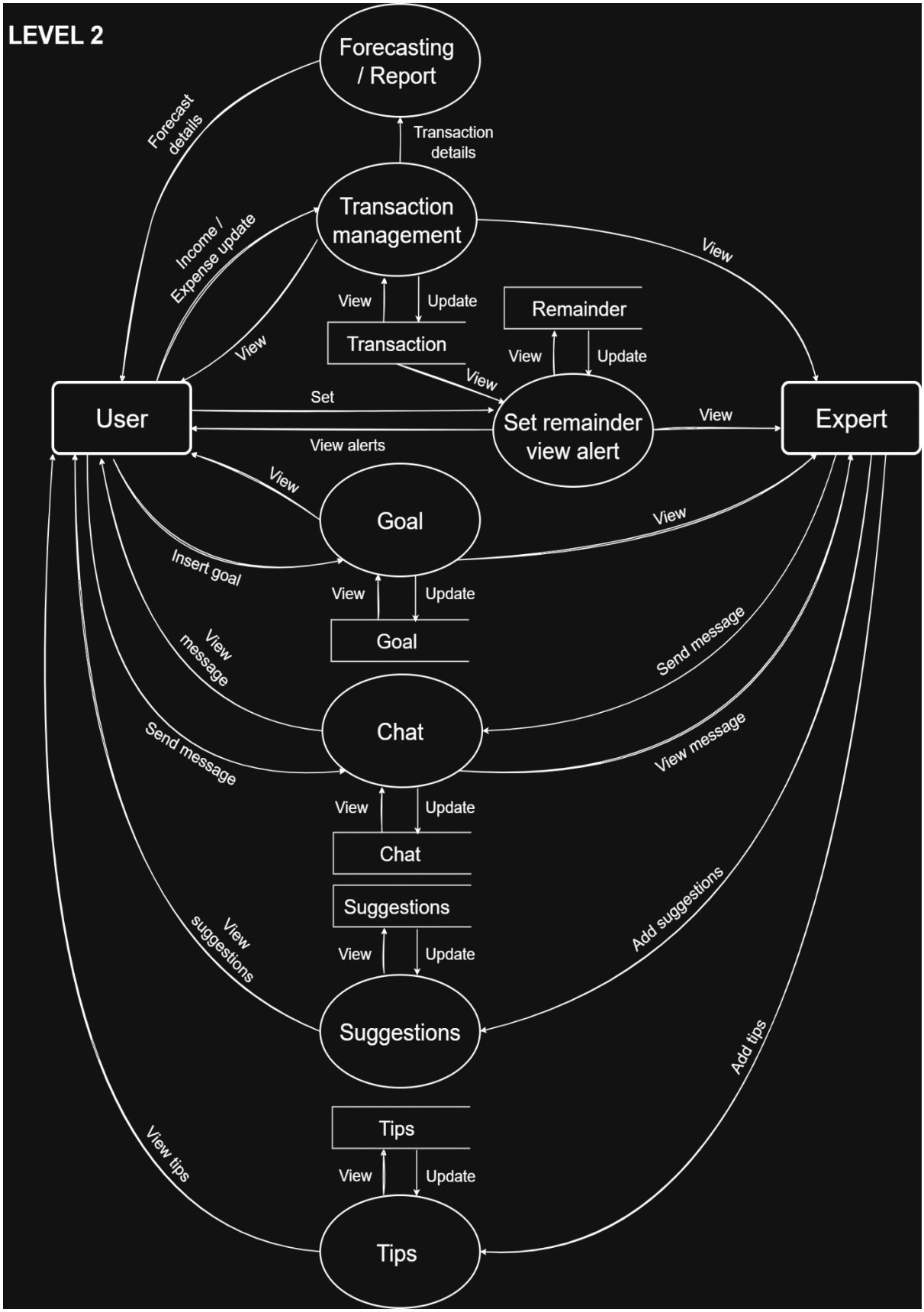

Figure 6.4: Level 2 Data Flow Diagram of BudgetBuddy System

6.7 Database Design

The database design of BudgetBuddy was developed to ensure efficient data storage, integrity, security, and scalability. The system uses a relational database structure implemented through Django's ORM, where data is organized into multiple interrelated tables. Each table represents a specific entity in the system, such as User, Expert, Transaction, Goal, Reminder, Tips, Suggestions, Feedback, Complaint, and Chat. Primary keys uniquely identify each record, and foreign keys are used to establish relationships between tables, ensuring referential integrity.

6.8 Database Tables

Auth_user Table

Table 6.2: Structure of Auth_User Table

Field Name	Data Type	Constraints
id	int	Primary Key
username	varchar	Not Null
password	varchar	Not Null

Expert Table

Table 6.3: Structure of Expert Table

Field Name	Data Type	Constraints
id	bigint	Primary Key
Name	varchar	Not Null
DOB	date	Not Null
Gender	varchar	Not Null
Photo	varchar	Not Null
Place	varchar	Not Null
Post	varchar	Not Null
Pin	int	Not Null
Phone_number	bigint	Not Null
Email	varchar	Not Null
LOGIN_id	int	Not Null

User Table

Table 6.4: Structure of User Table

Field Name	Data Type	Constraints
id	bigint	Primary Key
Name	varchar	Not Null
DOB	date	Not Null
Gender	varchar	Not Null
Photo	varchar	Not Null
Place	varchar	Not Null
Post	varchar	Not Null
Pin	int	Not Null
Phone_number	bigint	Not Null
Email	varchar	Not Null
LOGIN_id	int	Not Null

Feedback Table

Table 6.5: Structure of Feedback Table

Field Name	Data Type	Constraints
id	bigint	Primary Key
Feedback	varchar	Not Null
Date	date	Not Null
Expert_id	bigint	Not Null
User_id	bigint	Not Null

Compliant Table

Table 6.6: Structure of Complaint Table

Field Name	Data Type	Constraints
id	bigint	Primary Key
Compliant	varchar	Not Null
Date	date	Not Null
Reply	varchar	Not Null
User_id	bigint	Not Null

Tips Table

Table 6.7: Structure of Tips Table

Field Name	Data Type	Constraints
id	bigint	Primary Key
Tip	varchar	Not Null
Date	date	Not Null
Expert_id	bigint	Not Null
Details	varchar	Not Null

Transaction Table

Table 6.8: Structure of Transaction Table

Field Name	Data Type	Constraints
id	bigint	Primary Key
Exp_type	varchar	Not Null
Mode	varchar	Not Null
Amount	int	Not Null
Date	date	Not Null
Tan_type	varchar	Not Null
User_id	bigint	Not Null

Reminder Table

Table 6.9: Structure of Reminder Table

Field Name	Data Type	Constraints
id	bigint	Primary Key
Type	varchar	Not Null
Amount	int	Not Null
Details	varchar	Not Null
Date	date	Not Null
Reminder_date	date	Not Null
User_id	bigint	Not Null
Status	varchar	Not Null

Chat Table

Table 6.10: Structure of Chat Table

Field Name	Data Type	Constraints
id	bigint	Primary Key
Message	varchar	Not Null
Date	date	Not Null
Status	varchar	Not Null
From_id_id	int	Not Null
To_id_id	int	Not Null
Time	varchar	Not Null

Suggestions Table

Table 11: Structure of Suggestion Table

Field Name	Data Type	Constraints
id	bigint	Primary Key
Suggestion	varchar	Not Null
Date	date	Not Null
Expert_id	bigint	Not Null
User_id	bigint	Not Null

Password Table

Table 6.12: Structure of Password Table

Field Name	Data Type	Constraints
id	bigint	Primary Key
email	varchar	Not Null
otp	varchar	Not Null
Created_at	datetime	Not Null

Notification Table

Table 6.13: Structure of Notification Table

Field Name	Data Type	Constraints
id	bigint	Primary Key
Date	date	Not Null
User_id	bigint	Not Null

Goal Table

Table 6.14: Structure of Goal Table

Field Name	Data Type	Constraints
id	bigint	Primary Key
Amount	int	Not Null
Date	date	Not Null
Details	varchar	Not Null
User_id	bigint	Not Null

7. SYSTEM TESTING & IMPLEMENTATION

7.1 System Testing

The testing of the BudgetBuddy system was conducted through a structured process to ensure that all modules functioned correctly and met the project requirements. The testing process was divided into four main phases: unit testing, integration testing, system testing, and user acceptance testing. Each phase focused on verifying the functionality, performance, and interaction of system components. Multiple testing cycles were performed throughout development to identify and correct errors at different stages.

Unit Testing

Unit testing was carried out to verify the functionality of individual modules independently. Core modules such as user registration and login, transaction management, reminder setup, goal management, and expert tips were tested separately. The purpose of this phase was to ensure that each module worked correctly before integration with other components.

Integration Testing

Integration testing was carried out to ensure that the different modules of the system worked together correctly. In this project, two cycles of integration testing were performed. In the first cycle, the interaction between the mobile application, backend server, and database was tested to verify that data entered through the application was correctly stored and retrieved. In the second cycle, integrated modules such as transaction management, report generation, and chat communication between users and experts were tested together to ensure proper data flow and system interaction. The results confirmed that the modules were successfully integrated and communicated correctly without errors.

System Testing

System testing was conducted to evaluate the complete BudgetBuddy application as an integrated system. In this project, two cycles of system testing were performed. In the first cycle, the entire application was tested to verify that all features, including expense tracking, goal setting, reminders, and budget alerts, worked correctly together. In the second cycle, the

system was tested under different usage scenarios to ensure stability, performance, and proper functioning of all modules after integration. The results confirmed that the system operated reliably and met the functional requirements of the project.

Validation Testing

Validation testing was conducted to ensure that the system met the specified requirements and handled user inputs correctly. In this project**,** two cycles of validation testing were performed. In the first cycle, the system was tested with normal user inputs to verify that all forms accepted valid data and rejected incorrect entries. In the second cycle, boundary and incorrect input cases were tested to ensure that appropriate validation messages were displayed and that the system handled errors properly. The results confirmed that the validation rules were correctly applied and that the system responded appropriately to user inputs.

User Acceptance Testing

User acceptance testing was conducted to evaluate the usability and functionality of the system from the end-user perspective. In this project, one cycle of user acceptance testing was performed. During this cycle, a group of student users tested the application by performing tasks such as registering an account, adding transactions, setting reminders, and viewing reports. The feedback obtained from the users confirmed that the system was easy to use and met their expectations for managing personal expenses.

7.2 System Implementation

System implementation is the final phase in which the system design is converted into a working system. During this stage, the developed application is installed, tested, and made ready for use by the users. The three common implementation approaches are Parallel Conversion, Direct Conversion and Pilot Conversion.

In this project, the Direct Conversion approach was used. In this method, the existing manual process is completely replaced by the new system at a specific time. Since the application was newly developed, this approach was suitable for deploying the system without running the old and new systems simultaneously.

To ensure effective usage, user training and documentation were provided. A user manual was prepared, and users were given demonstrations and hands-on sessions to

understand features such as account registration, adding transactions, setting reminders, and viewing reports.

Appropriate coding practices were also followed. Each data item in the system was assigned a unique identification code to ensure proper organization and efficient data management.

8. SYSTEM MAINTENANCE

System maintenance refers to the activities performed after the deployment of a software system to ensure that it continues to function efficiently, securely, and reliably. In general, software maintenance can be categorized into several approaches such as corrective, adaptive, perfective, and preventive maintenance. Although these maintenance approaches are available, the BudgetBuddy project primarily adopted **Corrective Maintenance**, as the main focus after deployment was to identify and resolve any errors that might occur during system usage.

8.1 Corrective Maintenance

Corrective maintenance involves identifying and fixing bugs or errors that may appear after the system has been implemented. In the BudgetBuddy application, this includes resolving issues related to transaction entries, notification delivery, chat functionality, and report generation. These issues are identified through system monitoring, testing, and user feedback, and are corrected by updating the relevant modules of the system. Implementing corrective maintenance ensures that the application continues to operate reliably and provides a smooth user experience.

9. FUTURE ENHANCEMENT

Although BudgetBuddy successfully provides an efficient platform for managing personal expenses and budgeting, there are several improvements and additional features that can be incorporated in future versions to make the system more advanced, user-friendly, and intelligent.

1. **Bank Transaction Message Extraction Integration**
   In the current implementation, BudgetBuddy requires users to manually enter expense details. In the future, the system can be enhanced by integrating automatic extraction of transaction information from bank SMS alerts or email notifications. This would allow expenses to be recorded automatically, reducing user effort and improving accuracy.
2. **Automatic Expense Categorization Using AI**
   Machine learning techniques can be implemented to automatically classify expenses into categories such as food, travel, shopping, bills, etc., based on transaction descriptions or spending patterns.
3. **Cloud Synchronization and Multi-Device Support**
   Adding cloud storage support will allow users to access their expense data securely from multiple devices, ensuring data backup and availability anytime.
4. **Integration with Digital Payment Platforms**
   BudgetBuddy can be extended to connect with digital wallets and UPI platforms for seamless expense tracking directly from payment history.
5. **Personalized Financial Recommendations**
   The system can be improved to provide personalized saving tips and budget recommendations based on user spending trends and financial goals.
6. **Multi-User and Family Budget Management**
   Future enhancements may include support for multiple users or shared family budgeting features to manage household expenses collaboratively.

10. CONCLUSION

BudgetBuddy is an efficient and intelligent expense management system developed to support individuals in tracking, analyzing, and forecasting their financial activities. The project successfully provides a structured platform where users can record daily income and expenses, categorize spending, set financial goals, and generate detailed reports for better budgeting and decision-making.

A key highlight of the system is the integration of predictive expense forecasting, which helps users estimate future expenses using historical spending data. Additionally, BudgetBuddy includes real-time budget alerts and reminders, ensuring that users are notified when their expenses approach or exceed predefined limits. The inclusion of expert financial tips, suggestions, and chat support further enhances the system by guiding users toward better financial discipline.

The project also implements a well-defined module structure involving Admin, Expert, and User roles, making the system scalable and suitable for effective monitoring and support. Although the current version does not include automatic bank transaction message extraction, BudgetBuddy forms a strong foundation for future improvements in automation and financial data integration.

Overall, BudgetBuddy serves as a reliable and user-friendly solution that promotes smart financial planning, proactive expense control, and sustainable budgeting practices among individuals.

11. BIBLIOGRAPHY

Bibliography

Dreamcatcher45. (2021). *Wallet – Expense Tracker*. Retrieved from GitHub: https://github.com/dreamcatcher45/wallet

Nguyen, F. (2022). *MMAS Money Tracker*. Retrieved from GitHub: https://github.com/floranguyen0/mmas-money-tracker

Santos, T. (2021). *Finance App Flutter*. Retrieved from GitHub: https://github.com/td-santos/Finance_App_Flutter

Books/Article

->> Sommerville :Software Engineering

->> Marco. L. Napoli :Beginning Flutter

->> Thomas Bailey: Flutter for Beginners

Web Resources

->> https://www.flutter.dev/

->> https://github.com/

->> https://Codelabs.developers.com/

->> https://codelabs.developers.google.com

->> https://firebase.google.com/docs

->> https://dart.dev/guides/language/

->> https://gemini.google.com/

12. APPENDIX

12.1 Sample Screenshots

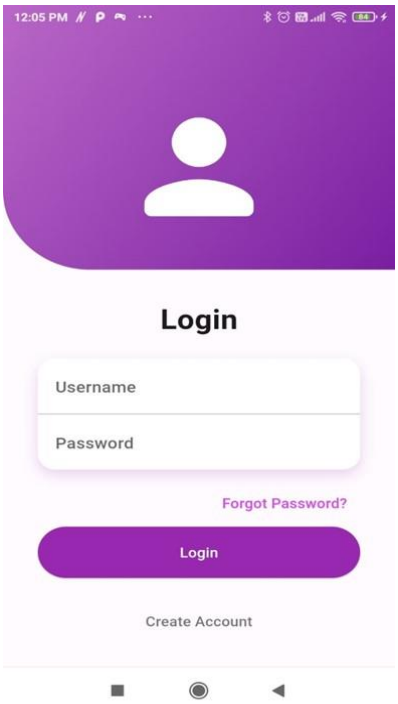

Figure 1: User Login Page

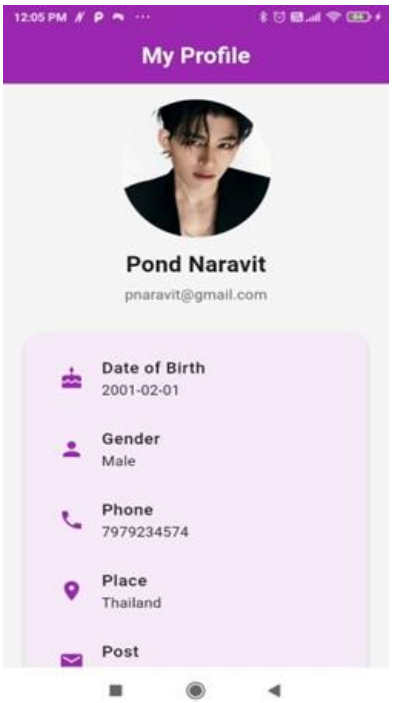

Figure 2: View Profile

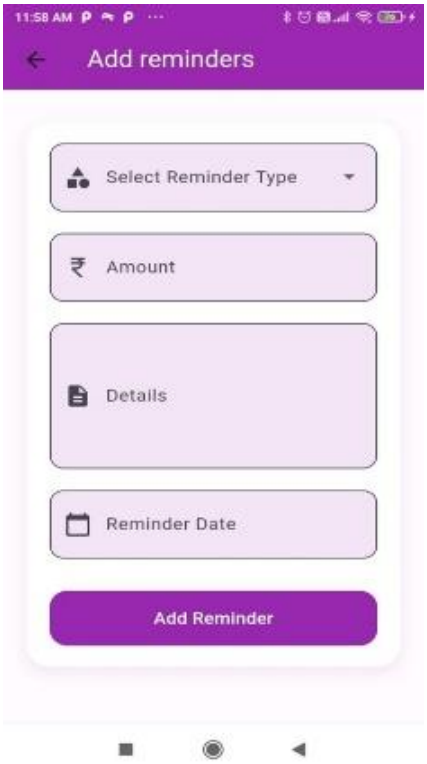

Figure 3: Add Remainder

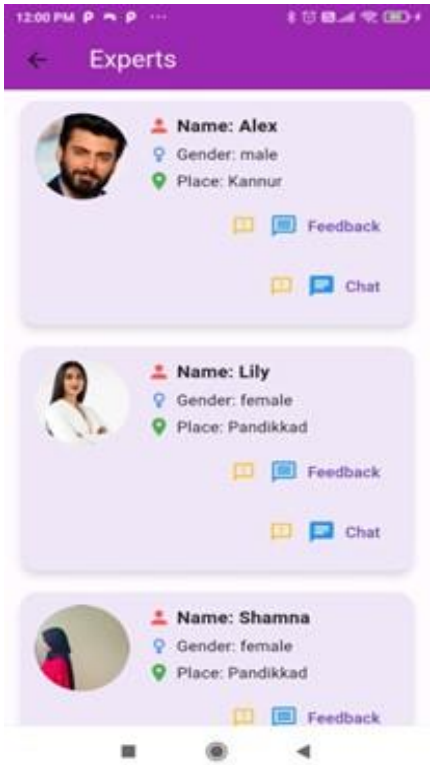

Figure 4: View Experts

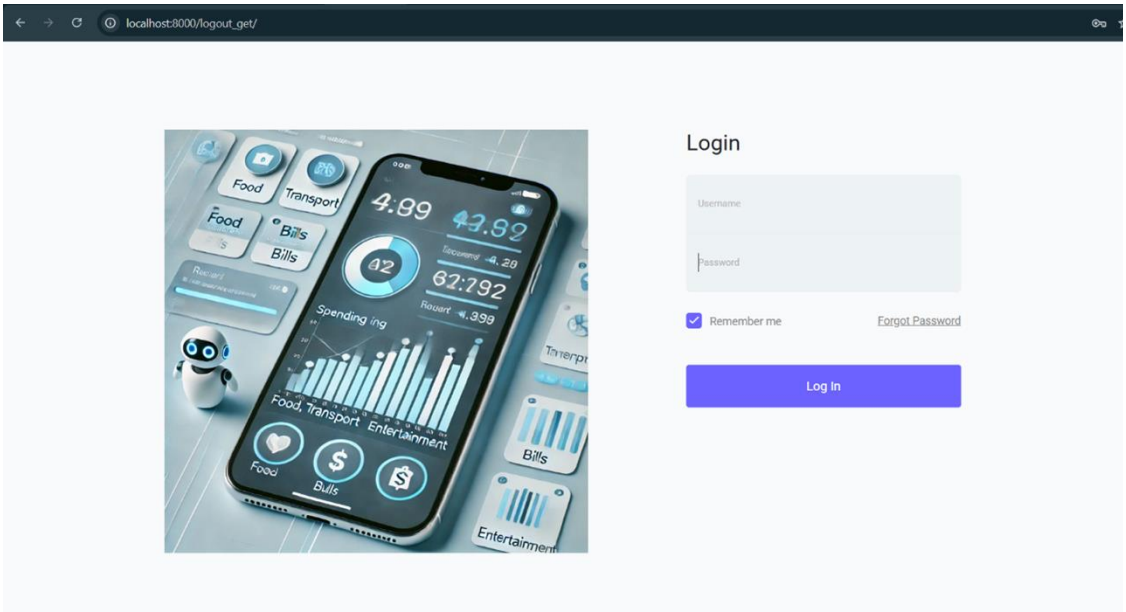

Figure 5: Admin & Expert Login Page

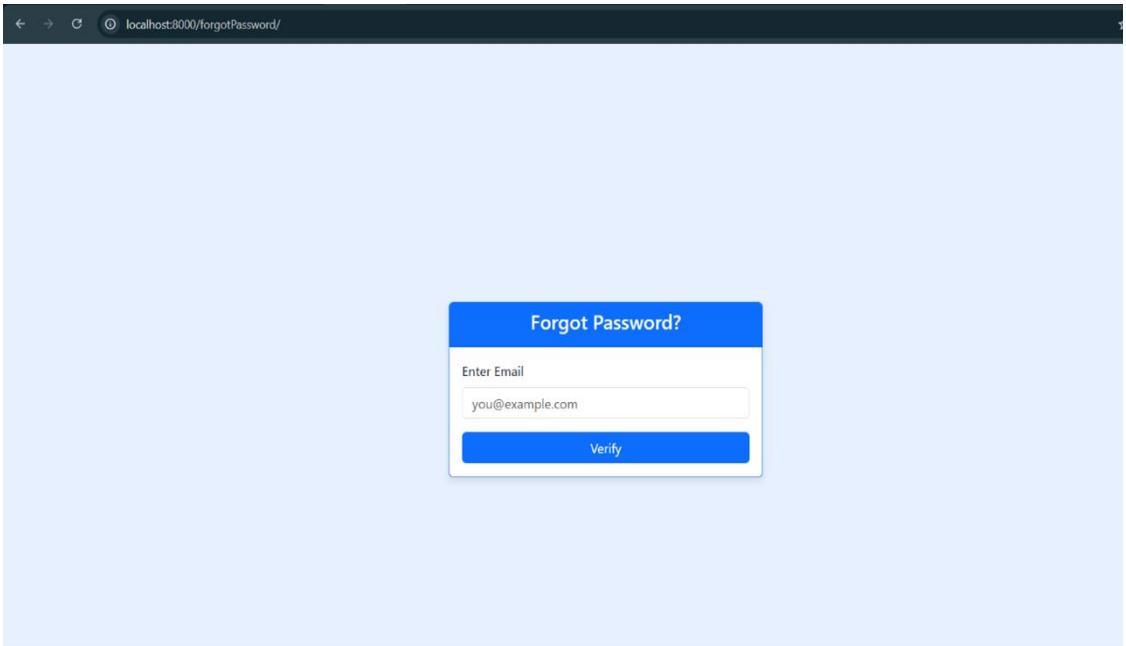

Figure 6: Forgot Password

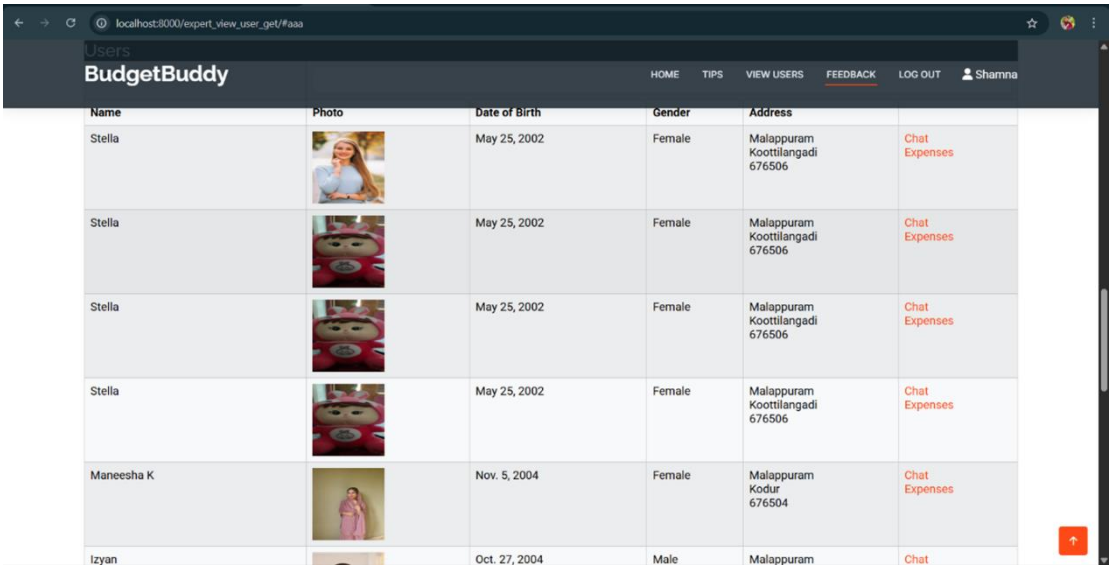

Figure 7: View Users

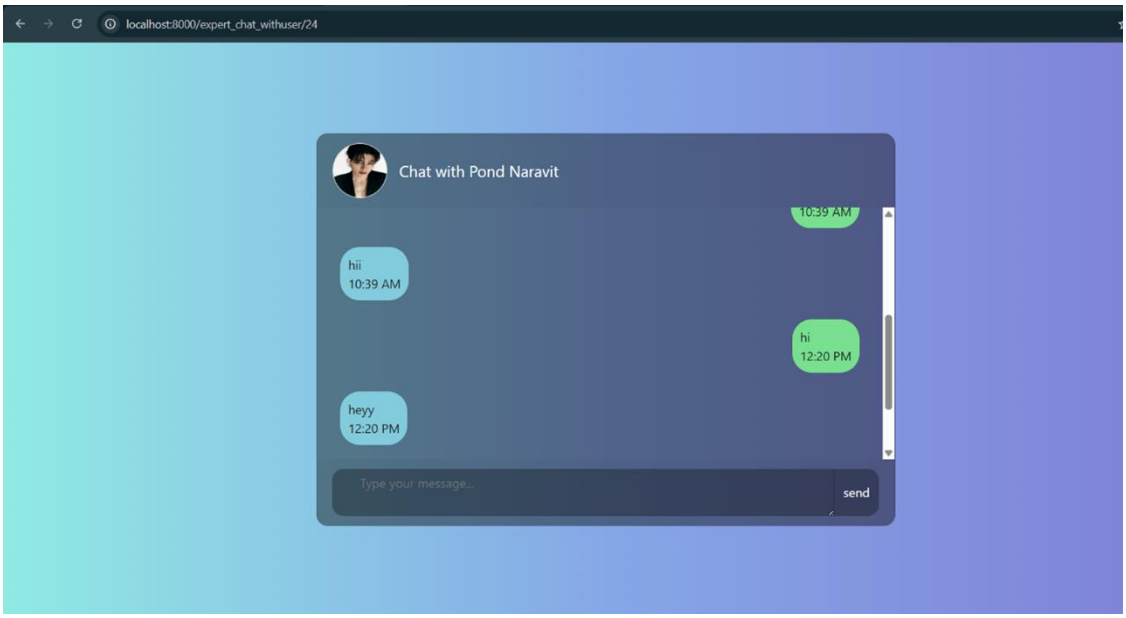

Figure 8: Expert Chat View

12.2 Sample Codes

Python Django:

```
# -------------------------------
# Set Goal Function
# -------------------------------
def set_goal(request):
    amount = request.POST['Amount']
    details = request.POST['Details']
    lid = request.POST['lid']

    goal = Goal()
    goal.Amount = amount
    goal.Date = datetime.now().today()
    goal.User = User_table.objects.get(LOGIN_id=lid)
    goal.Details = details
    goal.save()

    return JsonResponse({'status': 'ok'})


# -------------------------------
# Add Income / Expense Function
# -------------------------------
def add_income_expense(request):
    tan_type = request.POST['Tan_type']
    date = request.POST['Date']
    exp_type = request.POST['Exp_type']
    amount = request.POST['Amount']
    mode = request.POST['Mode']
    lid = request.POST['lid']

    transaction = Transaction()
    transaction.Exp_type = exp_type
    transaction.Date = date
    transaction.User = User_table.objects.get(LOGIN_id=lid)
    transaction.Mode = mode
    transaction.Amount = amount
    transaction.Tan_type = tan_type
    transaction.save()

    return JsonResponse({'status': 'ok'})


# -------------------------------
# View Feedback Function
# -------------------------------
def viewfeedback(request):
    lid = request.POST['lid']

    feedbacks = Feedback_table.objects.filter(User__LOGIN__id=lid)

    mdata = []
```

```
        for i in feedbacks:
            data = {
                'Feedback': i.Feedback,
                'User': i.User.Name,
                'Date': str(i.Date),
                'id': i.id,
                'Expert': i.Expert.Name
            }
            mdata.append(data)

    return JsonResponse({"status": "ok", "data": mdata})
```

Flutter:

```
-------------------------------
Home Page
-------------------------------
import 'dart:async';
import 'dart:convert';

import 'package:budget_buddy/prediction.dart';
import 'package:budget_buddy/tips.dart';
import 'package:budget_buddy/view complaints.dart';
import 'package:budget_buddy/view feedback.dart';
import 'package:budget_buddy/view reminder.dart';
import 'package:budget_buddy/view suggestions.dart';
import 'package:budget_buddy/view_experts.dart';
import 'package:budget_buddy/view_goal.dart';
import 'package:budget_buddy/view_report.dart';

import 'package:flutter/material.dart';
import
'package:flutter_local_notifications/flutter_local_notifications.dart';
import 'package:fluttertoast/fluttertoast.dart';

import 'package:shared_preferences/shared_preferences.dart';
import 'package:http/http.dart' as http';

import 'ViewProfile.dart';
import 'login.dart';
import 'manage income expense.dart';

class HomePage extends StatefulWidget {
  const HomePage({Key? key}) : super(key: key);

  @override
  State<HomePage> createState() => _HomePageState();
```

```
}

class _HomePageState extends State<HomePage> {

  // Index value for Bottom Navigation Bar
  int _selectedIndex = 0;

  @override
  void initState() {
    super.initState();

    // Start background notification task when homepage loads
    backgroundTask();
  }

  // Background task that checks notifications periodically
  void backgroundTask() {
    Timer.periodic(Duration(seconds: 20), (timer) {
      _send_data();
    });
  }

  // Function to display local notifications
  Future<void> showNotification(
      FlutterLocalNotificationsPlugin flutterLocalNotificationsPlugin,
      String msg) async {

    var androidDetails = AndroidNotificationDetails(
      'your_channel_id',
      'your_channel_name',
      importance: Importance.high,
      priority: Priority.high,
    );
```

```
  var platformDetails = NotificationDetails(android: androidDetails);

  await flutterLocalNotificationsPlugin.show(
    0,
    'Notification',
    msg,
    platformDetails,
  );
}

// Fetch notification details from backend server
void _send_data() async {

  SharedPreferences sh = await SharedPreferences.getInstance();

  // Getting stored server URL and login ID
  String url = sh.getString('url').toString();
  String lid = sh.getString('lid').toString();

  final urls = Uri.parse('$url/user_get_notification/');

  try {
    final response = await http.post(urls, body: {
      'lid': lid,
    });

    if (response.statusCode == 200) {

      // Decode response from server
      String status = jsonDecode(response.body)['status'];

      if (status == 'ok') {

        // Notification type and message from backend
```

```
        String type = jsonDecode(response.body)['type'].toString();
        String monthly_expense =
            jsonDecode(response.body)['monthly_expense'].toString();

        // Display reminder notification
        if (type == "rem") {
          final FlutterLocalNotificationsPlugin plugin =
              FlutterLocalNotificationsPlugin();
          await showNotification(plugin, monthly_expense);
        }

        // Display alert notification for spending limit
        else {
          final FlutterLocalNotificationsPlugin plugin =
              FlutterLocalNotificationsPlugin();
          await showNotification(
            plugin,
            'Alert! You've reached your monthly spending limit.',
          );
        }
      }
    }
  } catch (e) {
    Fluttertoast.showToast(msg: e.toString());
  }
}

// Grid menu items shown on dashboard
final List<Map<String, dynamic>> _items = [
  {"icon": Icons.tips_and_updates, "label": "Tips"},
  {"icon": Icons.currency_rupee, "label": "Manage Income/Expense"},
  {"icon": Icons.emoji_events, "label": "Set Goal"},
  {"icon": Icons.alarm, "label": "Reminder"},
  {"icon": Icons.description_rounded, "label": "Report"},
```

```
    {"icon": Icons.feedback, "label": "Feedback"},
    {"icon": Icons.comment, "label": "Complaint"},
    {"icon": Icons.settings_suggest, "label": "Suggesstions"},
    {"icon": Icons.person_2_rounded, "label": "Experts"},
    {"icon": Icons.trending_up, "label": "Smart Forecast"},
  ];

  // Bottom navigation bar actions
  void _onItemTapped(int index) {
    setState(() {
      _selectedIndex = index;

      // Navigate based on selected tab
      if (_selectedIndex == 0) {
        Navigator.push(
            context, MaterialPageRoute(builder: (context) => HomePage()));
      }
      else if (_selectedIndex == 1) {
        Navigator.push(
            context, MaterialPageRoute(builder: (context) =>
ViewProfile()));
      }
      else {
        Navigator.push(
            context, MaterialPageRoute(builder: (context) =>
MyLoginPage()));
      }
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(

      // AppBar Title
```

```
      appBar: AppBar(
        title: Text(
          "BudgetBuddy",
          style: TextStyle(color: Colors.white),
        ),
        backgroundColor: Colors.purple,
      ),

      // Main Dashboard Body
      body: Stack(
        children: [

          // Background Image for Home Screen
          Container(
            decoration: const BoxDecoration(
              image: DecorationImage(
                image: AssetImage("assets/bg.jpg"),
                fit: BoxFit.cover,
              ),
            ),
          ),

          // White overlay for readability
          Container(color: Colors.white.withOpacity(0.6)),

          // Grid menu for accessing features
          Center(
            child: GridView.builder(
              padding: const EdgeInsets.all(20),
              itemCount: _items.length,
                gridDelegate: const
SliverGridDelegateWithFixedCrossAxisCount(
                crossAxisCount: 2,
                crossAxisSpacing: 20,
```

```
                mainAxisSpacing: 20,
              ),
              itemBuilder: (context, index) {

                return GestureDetector(
                  onTap: () {

                    // Navigation based on selected menu item
                    switch (_items[index]['label']) {

                      case "Tips":
                        Navigator.push(context,
                            MaterialPageRoute(builder: (context) =>
tips()));
                        break;

                      case "Manage Income/Expense":
                        Navigator.push(
                            context,
                            MaterialPageRoute(
                                builder: (context) =>
                                    Manage_income_expense()));
                        break;

                      case "Set Goal":
                        Navigator.push(
                            context,
                            MaterialPageRoute(
                                builder: (context) => view_goal()));
                        break;

                       case "Reminder":
                        Navigator.push(
                            context,
                            MaterialPageRoute(
```

```
                builder: (context) => View_reminder()));
        break;

      case "Report":
        Navigator.push(
            context,
            MaterialPageRoute(
                builder: (context) => ViewReport()));
        break;

      case "Feedback":
        Navigator.push(
            context,
            MaterialPageRoute(
                builder: (context) => view_feedback()));
        break;

      case "Complaint":
        Navigator.push(
            context,
            MaterialPageRoute(
                builder: (context) => View_complaints()));
        break;

      case "Suggesstions":
        Navigator.push(
            context,
            MaterialPageRoute(
                builder: (context) => suggestions()));
        break;

      case "Experts":
        Navigator.push(
            context,
```

```
                MaterialPageRoute(
                    builder: (context) => view_expert()));
        break;

      case "Smart Forecast":
        Navigator.push(
            context,
            MaterialPageRoute(
                builder: (context) => ForecastPage()));
        break;
    }
  },

  // Menu UI Design
  child: Column(
    mainAxisSize: MainAxisSize.min,
    children: [
      CircleAvatar(
        radius: 35,
        backgroundColor: Colors.purple.withOpacity(0.2),
        child: Icon(
          _items[index]["icon"],
          color: Colors.purple,
          size: 32,
        ),
      ),
      const SizedBox(height: 8),
      Text(
        _items[index]["label"],
           style: const TextStyle(
            color: Colors.black, fontSize: 14),
      ),
    ],
  ),
```

```
                  );
                },
            ),
          ),
        ],
      ),

      // Bottom Navigation Bar
      bottomNavigationBar: BottomNavigationBar(
        backgroundColor: Colors.purple,
        currentIndex: _selectedIndex,
        selectedItemColor: Colors.white,
        unselectedItemColor: Colors.white70,
        onTap: _onItemTapped,
        items: const [
          BottomNavigationBarItem(icon: Icon(Icons.home), label: "Home"),
          BottomNavigationBarItem(icon: Icon(Icons.person), label:
"Profile"),
          BottomNavigationBarItem(icon: Icon(Icons.logout), label:
"Logout"),
        ],
      ),
    );
  }
}
```

Add Income and Expense

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:budget_buddy/register.dart';
import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
import 'package:fluttertoast/fluttertoast.dart';
```

```
import 'package:shared_preferences/shared_preferences.dart';
import 'package:intl/intl.dart';

import 'Newhome.dart';
import 'drawer.dart';
import 'manage income expense.dart';

void main() {
  runApp(myapp());
}

class myapp extends StatelessWidget {
  const myapp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: addexpense(),
    );
  }
}

class addexpense extends StatefulWidget {
  const addexpense({super.key});

  @override
  State<addexpense> createState() => _addexpenseState();
}

class _addexpenseState extends State<addexpense> {

  // Controllers for input fields
  TextEditingController dateController = new TextEditingController();
  TextEditingController typeController = new TextEditingController();
```

```
TextEditingController tanController = new TextEditingController();
TextEditingController amountController = new TextEditingController();
TextEditingController modeController = new TextEditingController();

// Function to select date using Date Picker
Future<void> _selectDate(BuildContext context) async {
  DateTime? pickedDate = await showDatePicker(
    context: context,
    initialDate: DateTime.now(),
    firstDate: DateTime(2000),
    lastDate: DateTime(2100),
  );

  // Format selected date and display it in TextField
  if (pickedDate != null) {
    String formattedDate = DateFormat('yyyy-MM-dd').format(pickedDate);
    setState(() {
      dateController.text = formattedDate;
    });
  }
}

// Dropdown values for Transaction Type
String? selectedValue1;
final List<String> items1 = ['Credit', 'Debit'];

// Dropdown values for Mode of Payment
String? selectedValue;
final List<String> items = ['Online', 'Offline'];

@override
Widget build(BuildContext context) {
  return Scaffold(
```

```
// Page Title
 appBar: AppBar(
  title: Text(
    "Add Expense or Income",
    style: TextStyle(color: Colors.white),
  ),
  backgroundColor: Colors.purple,
),

// Main Form Body
body: SingleChildScrollView(
  child: Column(
    children: [

      // Form Container with Styling
      Padding(
        padding: const EdgeInsets.all(20.0),
        child: Container(
          decoration: BoxDecoration(
            color: Colors.white,
            borderRadius: BorderRadius.circular(25),
            boxShadow: [
              BoxShadow(
                color: Colors.grey.withOpacity(0.2),
                blurRadius: 8,
                spreadRadius: 2,
                offset: const Offset(0, 5),
              ),
            ],
          ),

            // Input Form Fields
          child: Padding(
            padding: const EdgeInsets.all(20.0),
```

```
child: Column(
  children: [

    // Dropdown for selecting Credit or Debit
    DropdownButtonFormField<String>(
      value: selectedValue1,
      decoration: InputDecoration(
        prefixIcon: const Icon(Icons.swap_horiz),
        labelText: "Transaction Type",
        border: OutlineInputBorder(
            borderRadius: BorderRadius.circular(12)),
      ),
      items: items1.map((String item) {
        return DropdownMenuItem<String>(
          value: item,
          child: Text(item),
        );
      }).toList(),
      onChanged: (String? newValue) {
        setState(() {
          selectedValue1 = newValue;
        });
      },
    ),
    const SizedBox(height: 20),

    // Dropdown for selecting Online or Offline mode
    DropdownButtonFormField<String>(
      value: selectedValue,
      decoration: InputDecoration(
           prefixIcon: const Icon(Icons.payment),
        labelText: "Mode",
        border: OutlineInputBorder(
            borderRadius: BorderRadius.circular(12)),
```

```
    ),
      items: items.map((String item) {
      return DropdownMenuItem<String>(
        value: item,
        child: Text(item),
      );
    }).toList(),
    onChanged: (String? newValue) {
      setState(() {
        selectedValue = newValue;
      });
    },
  ),
  const SizedBox(height: 20),

  // Date input field with calendar picker
  TextFormField(
    readOnly: true,
    onTap: () => _selectDate(context),
    controller: dateController,
    decoration: InputDecoration(
      prefixIcon: const Icon(Icons.calendar_today),
      labelText: "Date",
      border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(12)),
    ),
  ),
  const SizedBox(height: 20),

  // Field for entering expense or income category/type
  TextFormField(
    controller: typeController,
    decoration: InputDecoration(
      prefixIcon: const Icon(Icons.category),
```

```
        labelText: "Expense / Income Type",
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(12)),
    ),
  ),
  const SizedBox(height: 20),

  // Field for entering transaction amount
  TextFormField(
    controller: amountController,
    keyboardType: TextInputType.number,
    decoration: InputDecoration(
      prefixIcon: const Icon(Icons.currency_rupee),
      labelText: "Amount",
      border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(12)),
    ),
  ),
  const SizedBox(height: 30),

  // Save button to submit transaction details
  SizedBox(
    width: double.infinity,
    height: 50,
    child: ElevatedButton(
      style: ElevatedButton.styleFrom(
        backgroundColor: Colors.purple,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(15),
        ),
        elevation: 5,
      ),
      onPressed: _send_data,
      child: const Text(
```

```
                          "Save",
                          style: TextStyle(
                            fontSize: 18,
                            fontWeight: FontWeight.w600,
                            color: Colors.white,
                          ),
                        ),
                      ),
                    ),
                  ],
                ),
              ),
            ),
          ),
        ],
      ),
    ),
  );
}

// Function to send transaction data to backend server
void _send_data() async {

  // Collect values entered by user
  String date = dateController.text;
  String amount = amountController.text;
  String type = typeController.text;

  SharedPreferences sh = await SharedPreferences.getInstance();

  // Fetch stored server URL and login ID
  String url = sh.getString('url').toString();
  String lid = sh.getString('lid').toString();
```

```
final urls = Uri.parse('$url/add_income_expense/');

try {
  final response = await http.post(urls, body: {
    'Date': date,
    'Exp_type': type,
    'Amount': amount,
    'Tan_type': selectedValue1,
    'lid': lid,
    'Mode': selectedValue,
  });

  if (response.statusCode == 200) {

    // Decode backend response
    String status = jsonDecode(response.body)['status'];

    if (status == 'ok') {
      Fluttertoast.showToast(msg: 'Transaction Added Successfully');

      // Redirect user back to Manage Income/Expense page
      Navigator.push(
        context,
        MaterialPageRoute(
          builder: (context) => Manage_income_expense(),
        ),
      );
    } else {
      Fluttertoast.showToast(msg: 'Failed to Add Transaction');
    }
  } else {
    Fluttertoast.showToast(msg: 'Network Error');
  }
} catch (e) {
```

```
      Fluttertoast.showToast(msg: e.toString());
     }
   }
}
```